

ŚCIAĞA REKRUTACYJNA

SQL · PostgreSQL · Docker · Kubernetes

Poziom Junior → Mid | pojęcia i definicje

► SQL & PostgreSQL

Słownik pojęć

POJĘCIE	CO TO JEST
SELECT	Pobierz dane z tabeli. Rozbudowujesz o WHERE, ORDER BY, LIMIT, DISTINCT.
WHERE	Filtruj wiersze PRZED zwróceniem wyników. Np. WHERE age > 18.
GROUP BY	Zgrupuj wiersze o tych samych wartościach, żeby użyć COUNT/SUM/AVG.
HAVING	Filtruje GRUPY po GROUP BY — jak WHERE, ale dla zagregowanych danych.
INNER JOIN	Zwraca wiersze mające dopasowanie w OBU tabelach. Najczęstszy JOIN.
LEFT JOIN	Wszystkie wiersze z lewej tabeli + pasujące z prawej (brak = NULL).
RIGHT JOIN	Jak LEFT JOIN, ale odwrotnie — wszystkie z prawej.
FULL OUTER JOIN	Wszystkie z obu tabel. Brak dopasowania po obu stronach = NULL.
CROSS JOIN	Iloczyn kartezjański — każdy z każdym. Rzadko używany celowo.
PRIMARY KEY	Unikalny identyfikator wiersza. Automatycznie NOT NULL i UNIQUE.
FOREIGN KEY	Klucz obcy — referencja do PK innej tabeli. Zapewnia integralność danych.
UNIQUE	Wartości w kolumnie muszą być unikalne. Dopuszcza NULL, może być wiele.
NOT NULL	Kolumna musi mieć wartość — zabrania NULL.
INDEX	Struktura (B-tree) przyspieszająca wyszukiwanie kosztem wolniejszego zapisu.
NULL	Brak wartości. Porównuj przez IS NULL / IS NOT NULL, nie = NULL.
ALIAS (AS)	Tymczasowa nazwa dla kolumny lub tabeli w zapytaniu. SELECT name AS imie.
SUBQUERY	Zapytanie SELECT w środku innego zapytania — w WHERE, FROM lub SELECT.
UNION / UNION ALL	Łączy wyniki dwóch SELECT. UNION usuwa duplikaty, UNION ALL nie.
TRANSACTION	Blok operacji: albo wszystkie się wykonają, albo żadna. BEGIN / COMMIT / ROLLBACK.
ACID	Atomicity, Consistency, Isolation, Durability — 4 gwarancje transakcji.
VIEW	Zapisane zapytanie SELECT jako wirtualna tabela. Nie przechowuje danych.
MATERIALIZED VIEW	Jak VIEW, ale wyniki zapisane na dysku. Szybszy odczyt, trzeba odświeżać.
CTE (WITH)	Tymczasowy wynik na początku zapytania: WITH x AS (SELECT ...). Poprawia czytelność.
WINDOW FUNCTION	Agregat BEZ zwijania wierszy. ROW_NUMBER, RANK, LAG, LEAD, SUM OVER...
EXPLAIN ANALYZE	Pokaż plan i czas wykonania zapytania. Kluczowe do optymalizacji.
SERIAL / IDENTITY	Auto-increment. SERIAL — skrót Postgres. IDENTITY — nowszy standard SQL.
TRUNCATE	Usuwa wszystkie wiersze szybko, bez logowania. Szybszy od DELETE.
CASCADE	Propaguje DELETE/DROP do powiązanych rekordów lub obiektów.
VACUUM	Odzyskuje miejsce po martwych wersjach wierszy (Postgres MVCC).

POJĘCIE	CO TO JEST
MVCC	Wiele wersji wiersza jednocześnie — transakcje nie blokują się wzajemnie.
DEADLOCK	Dwie transakcje czekają na siebie. Postgres wykrywa i ubija jedną.
JSONB	JSON przechowywany binarnie w Postgres. Szybszy, można indeksować (GIN).
PARTITIONING	Podział dużej tabeli na mniejsze partycje (np. po dacie). Szybsze zapytania.
N+1 PROBLEM	Pobierasz N obiektów i robisz N osobnych zapytań. Rozwiązanie: JOIN lub batch.

Pytania rekrutacyjne

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
DELETE vs TRUNCATE vs DROP?	DELETE — usuwa wiersze (można WHERE, ROLLBACK). TRUNCATE — wszystkie wiersze, szybko, bez logowania. DROP — usuwa całą tabelę ze strukturą.
WHERE vs HAVING?	WHERE filtruje wiersze przed grupowaniem. HAVING filtruje grupy po GROUP BY.
Jakie są rodzaje JOINów?	INNER (tylko pasujące), LEFT (wszystkie z lewej), RIGHT, FULL OUTER (wszystkie z obu), CROSS (każdy z każdym).
Co to indeks i kiedy go używać?	Przyspiesza wyszukiwanie. Używaj na kolumnach w WHERE i JOIN. Sprawdź EXPLAIN ANALYZE czy jest używany.
Co to N+1 problem?	Pobierasz N rekordów i dla każdego robisz osobne zapytanie = N+1 zapytań. Fix: JOIN lub batch load.
Co to transakcja?	Blok operacji: albo wszystkie się wykonają (COMMIT), albo żadna (ROLLBACK). Przykład: przelew bankowy.
Co to CTE?	WITH nazwa AS (SELECT...) — tymczasowy wynik dla czytelności. Może być rekurencyjne (hierarchie).
Co to Window Function?	Agregat bez zwijania wierszy. ROW_NUMBER() do rankingów, LAG() do różnic między wierszami, SUM OVER do sum narastających.
Jak optymalizujesz wolne zapytanie?	EXPLAIN ANALYZE → szukaj Sequential Scan → dodaj indeks. Sprawdź N+1. Unikaj SELECT *. Rozważ Materialized View.
JSONB vs JSON w Postgres?	JSON to surowy tekst. JSONB — binarny, szybszy odczyt, można indeksować. Prawie zawsze używaj JSONB.



Słownik pojęć

POJĘCIE	CO TO JEST
Konteneryzacja	Pakuje aplikację + zależności w izolowaną jednostkę działającą tak samo wszędzie.
Kontener vs VM	VM = cały OS (ciężki, GB, wolny start). Kontener = współdzieli jądro hosta (MB, sekundy).
Image (obraz)	Read-only szablon z warstwami. Jak klasa — z jednego image uruchamiasz wiele kontenerów.
Kontener	Uruchomiona instancja image + warstwa zapisu. Po usunięciu dane giną (bez volume).
Dockerfile	Plik z instrukcjami budowania image: FROM, RUN, COPY, ENV, EXPOSE, CMD/ENTRYPOINT.
Docker Hub	Publiczny rejestr obrazów — jak npm dla Dockera. Oficjalne obrazy: postgres, nginx, node...
Registry	Serwer przechowujący obrazy. Prywatne: AWS ECR, Google GCR, GitLab Registry, Harbor.
Layer (warstwa)	Każda instrukcja Dockerfile = warstwa. Warstwy są cache'owane — zmiana później nie inwaliduje wcześniejszych.
CMD	Domyślna komenda przy starcie. Można ją nadpisać przy docker run.
ENTRYPOINT	Zawsze się wykonuje przy starcie. CMD staje się jego argumentem. Użyj gdy kontener to „narzędzie”.
Volume	Przechowuje dane POZA kontenerem. Przeżywa usunięcie kontenera. Używaj dla baz i uploadów.
Bind Mount	Montuje katalog z HOSTA do kontenera. Zmiany widoczne od razu. Używany w developmencie.
Port mapping	-p 8080:80 — port 8080 hosta → port 80 kontenera. Bez tego kontener jest niedostępny z zewnątrz.
Docker Network	Wirtualna sieć dla kontenerów. Bridge (domyślna), Host (sieć hosta), None (brak sieci).
docker-compose	Definiuje wiele kontenerów w jednym pliku YAML. Całe środowisko uruchamiasz jedną komendą.
Multi-stage build	Budowanie w etapach — np. kompilacja w ciężkim obrazie, finał w małym. Efekt: obraz 10x mniejszy.
.dockerignore	Wyklucza pliki z build context (jak .gitignore). Dodaj node_modules, .git, .env.
docker run	Tworzy i uruchamia kontener. Ważne flagi: -d (tło), -p (porty), -e (env), -v (volume), --rm.
docker exec	Uruchamia komendę w działającym kontenerze. docker exec -it myapp bash — debugowanie.
Health check	Sprawdza czy aplikacja działa. Jeśli failuje X razy — kontener jest oznaczony jako unhealthy.
Non-root user	Dobra praktyka: USER appuser w Dockerfile. Kontener nie działa jako root — bezpieczniej.
Skanowanie obrazów	Sprawdzanie image pod kątem podatności (CVE). Narzędzia: Trivy, Snyk, Docker Scout.

Pytania rekrutacyjne

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
Kontener vs VM?	VM wirtualizuje cały OS — ciężki (GB), wolny start. Kontener współdzieli jądro — lekki (MB), start w sekundy. Kontenery często działają NA VM.
Image vs kontener?	Image = read-only szablon (klasa). Kontener = uruchomiona instancja z warstwą zapisu (obiekt). Po usunięciu kontenera dane giną, image zostaje.
CMD vs ENTRYPOINT?	CMD to domyślna komenda — można nadpisać przy docker run. ENTRYPOINT zawsze się wykonuje, CMD jest jego argumentem.
Po co są volume'y?	Dane w kontenerze giną po jego usunięciu. Volume przechowuje je poza cyklem życia kontenera. Obowiązkowe dla baz danych.
Co to multi-stage build?	Budowanie w etapach — pierwszy etap kompiluje (ciężki toolchain), drugi kopiuje tylko artefakt do małego obrazu. Mniejszy, bezpieczniejszy image produkcyjny.
Jak zmniejszyć rozmiar image?	Alpine jako baza, multi-stage build, .dockerignore, łącz RUN w jedną warstwę, usuwaj cache (apt clean), nie instaluj zbędnych narzędzi.
Jak kontenery się komunikują?	W tej samej sieci Docker po nazwie serwisu/kontenera (jak DNS). W docker-compose app łączy się do db:5432. Na zewnątrz tylko przez zmapowane porty.
Co to docker-compose?	Definiuje wiele kontenerów w jednym YAML. Opisujesz całe środowisko (app + baza + cache) i startujesz komendą docker compose up.
Jak zabezpieczyć Docker?	Non-root USER, małe bazowe obrazy (Alpine), skanowanie (trivy), .dockerignore, sekrety w runtime nie w image, read-only filesystem tam gdzie możliwe.

► Kubernetes

Słownik pojęć

POJĘCIE	CO TO JEST
Kubernetes (K8s)	Orkiestrator kontenerów. Automatyzuje wdrażanie, skalowanie, restartowanie na wielu maszynach.
Klaster	Zestaw maszyn (nodów) zarządzanych przez K8s. Control Plane + Worker Nodes.
Control Plane	Mózg klastra: API Server, etcd (stan klastra), Scheduler, Controller Manager.
Node (węzeł)	Maszyna (VM lub fizyczna) w klastrze. Worker Node uruchamia Pody.
Pod	Najmniejsza jednostka K8s. 1+ kontenerów dzielących sieć i storage. Tymczasowe IP — nie łącz się bezpośrednio.
ReplicaSet	Zapewnia że żądana liczba Podów działa. Jeśli Pod padnie — RS uruchomi nowy. Zarządzany przez Deployment.
Deployment	Opisuje pożądany stan aplikacji: ile replik, który image. Obsługuje rolling update i rollback.
StatefulSet	Jak Deployment, ale dla aplikacji STANOWYCH (bazy danych). Każda replika ma stałą tożsamość i własne PVC.
DaemonSet	Jeden Pod na każdym nodzie. Używany dla agentów: log collector, monitoring, network plugin.
Job	Uruchamia Pod do jednorazowego zadania i kończy gdy się powiedzie. Np. migracja bazy, import danych.
CronJob	Job według harmonogramu (cron). Np. codzienne backupy, raporty, czyszczenie danych.
Service	Stały punkt dostępu do Podów (Pody mają zmienne IP). Daje stałe DNS i rozkłada ruch.
ClusterIP	Domyślny typ Service. Dostępny TYLKO wewnątrz klastra. Używasz do komunikacji między serwisami.
NodePort	Udostępnia Service na porcie (30000-32767) każdego noda. Dostępny z zewnątrz. Głównie do testów.
LoadBalancer	Tworzy zewnętrzny load balancer w chmurze (AWS ELB, GCP LB). Daje publiczne IP. Każdy kosztuje.
Ingress	Routing HTTP/HTTPS z zewnątrz do serwisów. Po ścieżce URL lub domenie. Jeden LB dla wielu serwisów — tańsze niż LoadBalancer per serwis.
Ingress Controller	Implementacja Ingressa. Musisz zainstalować osobno. Najpopularniejsze: nginx-ingress, Traefik.
ConfigMap	Konfiguracja (niesekretna) jako pary klucz-wartość lub plik. Montuj jako env lub plik w Podzie.
Secret	Wrażliwe dane: hasła, tokeny, certyfikaty. Zakodowane base64 (nie szyfrowane!). Nigdy nie commituj do Gita.
Namespace	Wirtualna izolacja zasobów w klastrze. Dzielisz klaster między środowiska (dev, prod) lub teamy.
PersistentVolume (PV)	Zasób storage w klastrze — dysk w chmurze, NFS, lokalny dysk. Tworzony przez admina lub automatycznie.
PersistentVolumeClaim	Żądanie o storage przez aplikację: „chcę 10GB”. K8s dopasowuje do PV. Montujesz jak volume.
StorageClass	Typ storage (SSD, HDD). Umożliwia automatyczne tworzenie PV gdy aplikacja tworzy PVC.

POJĘCIE	CO TO JEST
Liveness Probe	Czy kontener ŻYJE? Jeśli failuje X razy → K8s restartuje. Np. HTTP GET /healthz.
Readiness Probe	Czy kontener GOTOWY na ruch? Jeśli failuje → usunięty z Service, ale nie restartowany.
Startup Probe	Dla wolno startujących apek — K8s nie zabija kontenera zanim nie skończy startu.
HPA	Horizontal Pod Autoscaler — skaluje liczbę replik na podstawie CPU/memory lub custom metryk.
VPA	Vertical Pod Autoscaler — dostosowuje requests/limits CPU i memory na podstawie historii.
Resource Requests	Gwarancja zasobów dla Poda — scheduler decyduje gdzie go umieścić. ZAWSZE ustawiaj.
Resource Limits	Maksimum zasobów. Przekroczenie CPU = throttling. Przekroczenie memory = OOM Kill.
RBAC	Kontrola dostępu. Role + RoleBinding = kto może co robić. Zasada najmniejszych uprawnień.
ServiceAccount	Tożsamość dla Podów w klastrze. Używasz do RBAC wewnątrz klastra (np. Pod czyta Sekrety).
Helm	Package manager dla K8s. Chart = szablony YAML + wartości. Ułatwia deploy i wersjonowanie.
kubectl	CLI do zarządzania klastrem: get, describe, apply, delete, logs, exec. Łączy przez kubeconfig.
Taint / Toleration	Taint na nodzie odpycha Pody. Toleration w Podzie pozwala na „zainfekowanym” nodzie działać.
Node Affinity	Reguły przyciągania Podów do wybranych nodów. Twarde (wymagane) lub miękkie (preferowane).
Pod Anti-Affinity	Odpycha repliki od siebie — np. nie uruchamiaj dwóch replik na tym samym nodzie (HA).
Init Container	Kontener uruchamiany PRZED głównymi. Musi się skończyć sukcesem. Np. czekaj na bazę, migruj.
Rolling Update	Domyślna strategia aktualizacji — stopniowo zastępuj stare Pody nowymi, bez downtime.
etcd	Baza stanu całego klastra. Utrata etcd = utrata klastra. W produkcji zawsze rób backup!
Label	Para klucz-wartość przypisana do zasobu (Pod, Node...). Np. app=frontend, env=prod. Podstawa organizacji i selekcji.
Selector	Filtruje zasoby po labelach. Service używa selectora żeby wiedzieć do których Podów kierować ruch. Deployment używa go do zarządzania Podami.
Annotation	Para klucz-wartość jak label, ale NIE do selekcji. Służy do metadanych: wersja, autor, URL dokumentacji. Może być długa.

Pytania rekrutacyjne

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
Po co jest Kubernetes?	Gdy masz dużo kontenerów na wielu maszynach — K8s automatyzuje uruchamianie, restartowanie crashów, skalowanie pod obciążenie i aktualizacje bez downtime.

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
Pod vs kontener?	Kontener = izolowany proces. Pod = opakowanie na 1+ kontenerów, które zawsze działają razem i dzielą sieć. Zazwyczaj 1 kontener = 1 Pod.
Deployment vs StatefulSet?	Deployment dla bezstanowych aplikacji (serwery HTTP) — każda replika identyczna. StatefulSet dla stanowych (bazy danych) — stała tożsamość, własny storage, kolejność startu.
Do czego służy Service?	Pody mają zmienne IP. Service daje stałe DNS i rozkłada ruch. ClusterIP — wewnątrz klastra. NodePort — przez port noda. LoadBalancer — zewnętrzny LB w chmurze.
Po co Ingress skoro jest LoadBalancer?	LoadBalancer = osobne IP i koszt dla każdego serwisu. Ingress = jeden LB dla całej domeny, routing po URL/domenie do różnych serwisów. Tańsze i wygodniejsze.
ConfigMap vs Secret?	ConfigMap — niesekretna konfiguracja (endpointy, feature flags). Secret — wrażliwe dane (hasła, tokeny). Secret jest base64, nie szyfrowany — do prawdziwego szyfrowania używaj Vault.
Liveness vs Readiness Probe?	Liveness: czy żyje — jeśli nie, K8s restartuje. Readiness: czy gotowy na ruch — jeśli nie, usunięty z Service bez restartu. Zawsze ustawiaj oba.
Job vs Deployment?	Deployment działa ciągle (serwer). Job uruchamia się, robi zadanie i kończy (migracja bazy, import). CronJob = Job według harmonogramu.
Co to resource requests i limits?	Requests = gwarancja (scheduler decyduje gdzie umieścić Pod). Limits = maksimum (CPU throttling lub OOM Kill). Zawsze ustawiaj oba, inaczej scheduler działa losowo.
PV vs PVC?	PV to kawałek storage w klastrze. PVC to żądanie od aplikacji („chcę 10GB”). K8s dopasowuje PVC do PV. Aplikacja używa PVC i nie wie nic o szczegółach storage.
Co to RBAC?	Kontrola kto może co robić w klastrze. Role definiuje uprawnienia (np. get pods). RoleBinding przypisuje je do użytkownika lub ServiceAccount. Zasada: najmniejsze niezbędne uprawnienia.
Co to Helm?	Package manager dla K8s — zamiast wielu plików YAML masz Chart z szablonami i wartościami. Jeden helm install zamiast kubectl apply 20 plików. Ułatwia wersjonowanie i rollback.
Co to Label i Selector?	Label to para klucz-wartość na zasobie (np. app=api, env=prod). Selector filtruje zasoby po labelach — Service kieruje ruch do Podów pasujących do selectora. To klej łączący zasoby K8s.

► Git & CI/CD

Słownik pojęć

POJĘCIE	CO TO JEST
commit	Zapisuje snapshot zmian w lokalnym repo. <code>git commit -m "opis"</code> . Atomowy, logiczny, opisowy.
branch	Lekki wskaźnik na commit. Tworzysz gałąź na każdy feature/fix. <code>git checkout -b feature/login</code> .
merge	Łączy dwie gałęzie. Fast-forward (brak rozgałęzienia) lub merge commit (historia zachowana).
rebase	Przenosi commity na nową bazę — liniowa historia. Nie używaj na gałęziach współdzielonych!
cherry-pick	Aplikuje pojedynczy commit z innej gałęzi. <code>git cherry-pick</code> . Przydatne do hotfixów.
stash	Odkłada niezacommitowane zmiany na stos. <code>git stash</code> / <code>git stash pop</code> . Przydatne przy przełączaniu branchy.
revert	Tworzy nowy commit cofający zmiany. Bezpieczny — nie przepisuje historii. Używaj na main/master.
reset	Cofa HEAD do wskazanego commita. <code>--soft</code> (zmiany w staged), <code>--mixed</code> (unstaged), <code>--hard</code> (UWAGA: usuwa zmiany).
.gitignore	Lista plików/katalogów ignorowanych przez Git. Dodaj: <code>node_modules/</code> , <code>.env</code> , <code>*.log</code> , <code>dist/</code> .
Pull Request (PR)	Prośba o merge gałęzi. Miejsce code review, dyskusji i CI. Standardowy workflow zespołowy.
git flow	Model branchowania: main, develop, feature/*, release/*, hotfix/*. Sprawdza się w release-based projektach.
trunk-based dev	Krótkożyjące branche, częsty merge do main. Wymaga feature flags. Lepszy dla CI/CD.
tag	Stały wskaźnik na commit — używany do wersjonowania (v1.2.0). <code>git tag -a v1.0 -m "release"</code> .
HEAD	Wskaźnik na aktualny commit/branch. Detached HEAD = wskazuje bezpośrednio na commit.
origin	Domyślna nazwa zdalnego repo (np. GitHub). <code>git push origin main</code> = wyślij na GitHub.
conflict	Dwie gałęzie zmieniły ten sam fragment pliku. Git oznacza <<<<<<, musisz ręcznie rozwiązać.
Jenkins	Najpopularniejszy open-source serwer CI/CD. Pipelines w Jenkinsfile (Groovy). Ogromny ekosystem pluginów. Hostowany samodzielnie — pełna kontrola, ale wymaga utrzymania.
Jenkinsfile	Plik w repo definiujący pipeline. Declarative (prostszy, zalecany) lub Scripted (Groovy, elastyczniejszy). Wersjonowany razem z kodem — Pipeline as Code.
Pipeline (Jenkins)	Sekwencja etapów (stages): Checkout → Build → Test → Deploy. Każdy stage ma kroki (steps). Wizualizacja w Blue Ocean UI.
Agent	Maszyna wykonująca pipeline. <code>agent any</code> = dowolny dostępny. <code>agent { docker { image "node:18" } }</code> = uruchom w kontenerze. Master tylko koordynuje.
Stage / Step	Stage = logiczny etap (np. "Test"). Step = pojedyncza akcja w stage (np. <code>sh "npm test"</code>). Nieudany step → pipeline się zatrzymuje.
Credentials	Bezpieczne przechowywanie sekretów w Jenkinsie. Wstrzykujesz przez <code>withCredentials()</code> lub <code>environment {}</code> . Nigdy nie hardkoduj haseł w Jenkinsfile.

POJĘCIE	CO TO JEST
Webhook	GitHub/GitLab wywołuje Jenkinsa przy push/PR. Pipeline startuje automatycznie. Alternatywa: polling SCM (gorsze — odpytuje co X minut).
Artifact (Jenkins)	Plik wynikowy buildu (np. .jar, .zip, obraz Docker). archiveArtifacts przechowuje go w Jenkinsie. Późniejsze stage'y mogą go pobrać.
Blue Ocean	Nowoczesny UI dla Jenkinsa. Czytelna wizualizacja pipeline'ów, gałęzi i PR. Wbudowany edytor pipeline'ów.
Shared Library	Wspólny kod Groovy współdzielony między Jenkinsfilami. Unikasz duplikacji pipeline'ów między projektami. Wersjonowane w osobnym repo.

Pytania rekrutacyjne

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
merge vs rebase?	Merge zachowuje historię rozgałęzień (merge commit). Rebase przepisuje historię — liniowa, czytelna. Rebase lokalnie, merge do main.
git reset vs git revert?	reset przepisuje historię (niebezpieczny na shared branchy). revert tworzy nowy commit cofający — bezpieczny zawsze.
Co to Pull Request?	Propozycja merge gałęzi. Miejsce code review, uruchomienia CI i dyskusji. Merge po aprobacie recenzentów.
Jak wygląda dobry commit?	Atomowy (jedna zmiana), opisowy (co i dlaczego), w trybie rozkazującym: „Add login validation” nie „Added...”. Nie commituj zepsutego kodu.
git stash do czego?	Odkładasz niezacommitowane zmiany żeby przełączyć branch lub pull'nąć zmiany. git stash pop przywraca.
Co to CI/CD?	CI (Continuous Integration) = automatyczne testy i build przy każdym push. CD (Continuous Delivery/Deployment) = automatyczny deploy na środowisko po pozytywnym CI.
Jakie znasz narzędzia CI/CD?	GitHub Actions, GitLab CI, Jenkins, CircleCI, ArgoCD (GitOps dla K8s). W pipeline: build → test → lint → deploy.
Co to GitOps?	Infrastruktura i konfiguracja jako kod w Git. ArgoCD synchronizuje stan klastra K8s z repo. Pull zamiast push — bezpieczniejsze.
Jak obsługujesz sekrety w CI/CD?	Nigdy w kodzie ani YAML. Używaj: GitHub Secrets, GitLab CI Variables, HashiCorp Vault, AWS Secrets Manager. Wstrzykuj w runtime jako env.
Co sprawdzasz w code review?	Logika i poprawność, testy (czy są, czy sensowne), czytelność, bezpieczeństwo (SQL injection, sekrety), wydajność (N+1, indeksy).
Co to Jenkins i do czego służy?	Open-source serwer CI/CD. Przy każdym push uruchamia pipeline: build → test → deploy. Konfigurowany przez Jenkinsfile w repo (Pipeline as Code). Ogromny ekosystem pluginów.
Co to Jenkinsfile?	Plik w repo opisujący pipeline w Groovy. Declarative pipeline (zalecany) — czytelniejszy, prostszy. Scripted — pełny Groovy, elastyczniejszy. Wersjonowany razem z kodem.
Jenkins vs GitHub Actions?	Jenkins: self-hosted, pełna kontrola, wymaga utrzymania infrastruktury, ogromna liczba pluginów. GitHub Actions: w chmurze, zero utrzymania, świetna integracja z GH, prostszy YAML. Oba robią CI/CD — wybór zależy od infrastruktury.

PYTANIE REKRUTERA	DOBRA ODPOWIEDŹ
Jak przechowujesz sekrety w Jenkinsie?	Credentials Store w Jenkinsie — wstrzykujesz przez <code>withCredentials()</code> lub <code>environment {}</code> . Nigdy nie hardkodujesz haseł w <code>Jenkinsfile</code> . Na produkcji integracja z Vault.

► Wskazówki na rozmowie

Nie znasz odpowiedzi

Powiedz wprost: „Nie pamiętam szczegółów, ale podszedłbym do tego tak...” i opisz rozumowanie. Logiczne myślenie > wykuta definicja.

SQL — wspomnij zawsze

EXPLAIN ANALYZE przy optymalizacji. N+1 przy ORM. Transakcje gdy dane krytyczne. Indeksy — kiedy pomagają, kiedy szkodzą.

Docker — wspomnij zawsze

Multi-stage builds, małe obrazy, .dockerignore, non-root USER, health check, docker-compose do local devu.

K8s — wspomnij zawsze

Resource requests/limits, liveness + readiness probes, RBAC, Ingress zamiast wielu LoadBalancerów, Namespace do izolacji.

Pokazuj praktykę

„W projekcie X użyłem Y bo...” — konkrety > teoria. Brak komercyjnego? Opisz projekt osobisty lub naukowy.

Strukturyzuj odpowiedź

Definicja → przykład użycia → kiedy NIE używać. Np. „StatefulSet to... używamy gdy... zamiast Deploymentu bo...”

Pytaj na końcu

Zapytaj o stack, wielkość klastra K8s, CI/CD pipeline. Pokazujesz zainteresowanie i sprawdzasz czy projekt jest dla Ciebie.